LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

14

Linux Internals and Kernel Fundamentals

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Introduction
02 Kernel Architecture
03 System Calls
04 Process Creation
05 Threads and Context Switching
06 The Linux Scheduler
07 Kernel Modules
08 /proc and /sys
09 File Descriptors and Limits
10 Kernel Parameters: sysctl
11 Tracing with strace
12 PID 1
13 Troubleshooting Scenarios
14 Internals Cheat Sheet
15 Best Practices
16 Interview Q&A
17 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

CHAPTER OBJECTIVE

Understand what happens inside Linux when applications run: how they request kernel services through system calls, how processes and threads are created and scheduled, how kernel modules and virtual filesystems work, and how file descriptors and resource limits are enforced. Learn to investigate low-level problems using `/proc`, `/sys`, `sysctl`, `lsof` and `strace`.

1 Introduction

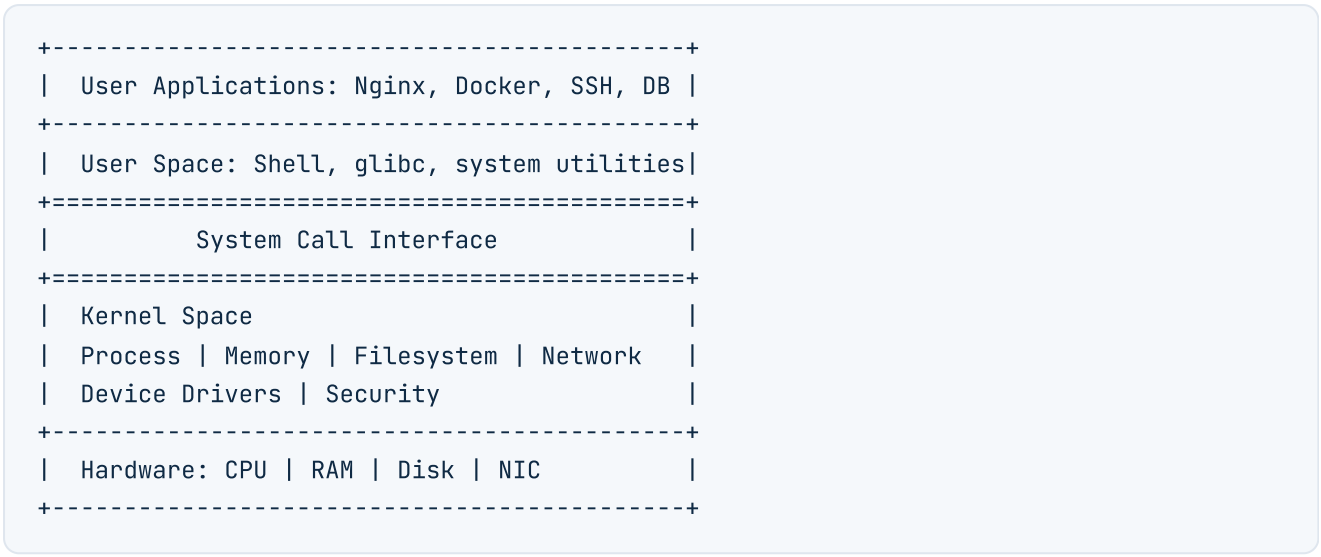
Earlier chapters covered administration, processes, services, networking, storage, security, automation and performance. This chapter looks at what happens **internally** when Linux runs commands, creates processes, manages hardware and talks to the kernel.

A DevOps engineer does not need to be a kernel developer, but internals knowledge helps answer questions such as:

- Why does a process receive `Permission denied` when the permissions look correct?
- How does Linux create a new process?
- Why can one process open a file but another cannot?
- What happens when a service reaches its open-file limit?
- How do kernel modules and drivers work?

2 Linux Kernel Architecture

The kernel manages communication between applications and hardware. Applications never access hardware directly; they request services from the kernel through the **system call interface**. Chapter 1 introduced the kernel's responsibilities and the user space vs kernel space split.



User Space	Kernel Space
Runs applications with limited privileges	Runs the kernel with full hardware access
Cannot execute privileged operations directly	Performs privileged operations on behalf of applications
Examples: Nginx, Bash, Python	Examples: scheduler, memory manager, drivers

Most applications do not make system calls directly. They call library functions in **glibc** (such as `fopen()`), which make the system calls for them.

> INTERVIEW LINE
>
> **The kernel provides controlled access to hardware and system resources while isolating applications from one another.**

3 Understanding System Calls

A system call is how a user-space program asks the kernel to do something it cannot do itself, such as open a file, create a process or send network data.

System call	Purpose
`openat()`	Open a file (modern replacement for `open()`)
`read()` / `write()`	Read or write data
`close()`	Close a file descriptor
`stat()`	Get file metadata
`fork()` / `clone()`	Create a process or thread
`execve()`	Run a new program
`wait4()` / `waitpid()`	Wait for a child process
`socket()` / `connect()`	Create a socket / open a connection
`mmap()`	Map files or memory into the address space

Example: Reading a File

1. The application calls `openat()` to open the file.
2. The kernel checks permissions (and SELinux/AppArmor) and locates the file.
3. The kernel returns a **file descriptor**, a small integer such as `3`.
4. The application calls `read()` with that descriptor.
5. The kernel returns the data and, later, the application calls `close()`.

File descriptors `0`, `1` and `2` are always stdin, stdout and stderr (see Chapter 11 for redirection). New files start at descriptor 3.

> **INTERVIEW NOTE**
>
> In `strace` output you will usually see `openat()`, not `open()`. Modern glibc uses `openat()` for almost all file opens.

4 Process Creation: fork(), exec() and wait()

Linux creates processes in two steps, then cleans up with a third. Chapter 1 showed how the shell runs `ls` this way.

fork()

Creates a **child** process that is a copy of the **parent**. Linux uses **copy-on-write**: parent and child share the same memory pages until one of them writes, and only then is that page copied. This makes `fork()` fast even for large processes.

exec()

The exec family (`execve()`) **replaces** the current program with a new one. It does not create a new process; the PID stays the same.

wait()

The parent calls `wait()` to collect the child's exit status. If the parent never does this, the finished child remains a **zombie** (see Chapter 5).

```
Parent (bash, PID 905)
      |
      | fork()          → child created (PID 910, copy of bash)
      v
Child (PID 910)
      |
      | execve("ls")    → same PID, now running ls
      v
ls runs and calls exit()
      |
      | wait()          → parent collects exit status (no zombie)
      v
Parent continues
```

Process Relationships

Every process has a PID and a parent (PPID). Chapter 5 covered `ps -ef` and `pstree`. Two more IDs matter for job control and signals:

ID	Meaning
PGID (process group)	A pipeline such as `cmd1 | cmd2` forms one group; Ctrl+C signals the whole group
SID (session)	All processes started from one login terminal; closing it can send SIGHUP to the session

```
ps -o pid,ppid,pgid,sid,uid,cmd -p 910
pstree -p
```

DEVOPS USE CASE

Killing a parent does not automatically kill its children; they become orphans adopted by PID 1. When a deployment script leaves stray child processes behind, use `pstree -p` or the PGID to find and stop the whole group, for example `kill -- -PGID`.

5 Threads and Context Switching

A thread is an execution unit inside a process. Threads in the same process share memory, open file descriptors and the address space, but each has its own stack and CPU state. Chapter 1 compared processes and threads; Linux schedules threads, not processes.

```
# All threads on the system
ps -eLf

# Threads of one process
ps -T -p 1234
top -H -p 1234

# Thread count of a process
grep Threads /proc/1234/status
```

Context Switching

A context switch happens when the CPU stops running one task, saves its state, and loads another. It enables multitasking, but excessive switching wastes CPU time.

```
# System-wide context switches per second (cs column)
vmstat 1

# Per-process: voluntary vs involuntary switches
grep ctxt /proc/1234/status
```

Type	Meaning
Voluntary	The task gave up the CPU because it was waiting (I/O, lock, sleep)
Involuntary	The scheduler took the CPU away because its time slice ended; high counts suggest CPU contention

6 The Linux Scheduler

The scheduler decides which runnable thread gets CPU time next. For normal workloads, it aims to share CPU fairly between tasks, weighted by their nice value. Nice and renice were covered in Chapter 5.

```
# Nice value (NI) and priority (PRI)
ps -eo pid,ni,pri,comm
```

CPU Affinity

CPU affinity restricts which CPU cores a process may run on.

```
# Show the CPUs a process may use
taskset -cp 1234

# Start a command pinned to CPU 0
taskset -c 0 ./application

# Change a running process to CPUs 0 and 1
sudo taskset -cp 0,1 1234
```

> **IMPORTANT**
>
> Nice values and CPU affinity influence scheduling, but they do not guarantee CPU time or set hard limits. For hard CPU limits, use cgroups, for example `CPUQuota=` in a systemd unit or container CPU limits (see Chapter 15).

7 Kernel Modules and Device Drivers

Kernel modules are pieces of code that can be loaded into the running kernel without rebuilding or rebooting it. Most hardware drivers, filesystem support and network features are modules. Chapter 8 introduced `lsmod`, `modinfo` and `modprobe`.

```
# Loaded modules
lsmod

# Module details (file, dependencies, parameters)
modinfo overlay

# Load / unload by name (resolves dependencies)
sudo modprobe overlay
sudo modprobe -r overlay

# Insert / remove a specific module file (no dependency handling)
sudo insmod ./mymodule.ko
sudo rmmod mymodule

# Load a module automatically at boot
echo "overlay" | sudo tee /etc/modules-load.d/overlay.conf
```

modprobe	insmod
Loads a module by name from `/lib/modules/$(uname -r)/`	Inserts a module from an exact file path
Automatically loads dependencies first	Fails if dependencies are not already loaded
Preferred for normal administration	Used for testing or custom modules

> **WARNING**
>
> Do not unload kernel modules on a production server without understanding what depends on them. Removing a module used by active storage, network or filesystem operations can crash services or the whole system.

8 The /proc and /sys Filesystems

Both are virtual filesystems generated by the kernel in memory. Nothing in them is stored on disk.

/proc – Processes and Kernel State

Path	Purpose
`/proc/cpuinfo`	CPU details
`/proc/meminfo`	Memory details (see Chapter 13)
`/proc/loadavg`	Load average
`/proc/mounts`	Mounted filesystems
`/proc/PID/`	One directory per process: `status`, `limits`, `fd/`, `cmdline`, `environ`
`/proc/sys/`	Runtime kernel parameters, managed with `sysctl`

/sys – Devices and Drivers

```
# Block devices
ls /sys/block/

# Network interfaces
ls /sys/class/net/

# Link speed of a network interface (Mb/s)
cat /sys/class/net/eth0/speed
```

/proc	/sys
Process information and kernel state	Devices, drivers and kernel objects
Has per-process directories	Organized by device type and bus
Holds runtime kernel parameters in `/proc/sys`	Exposes device and driver attributes

9 File Descriptors and Resource Limits

Every process has a file descriptor table. Each open file, socket, pipe or device uses one descriptor, and the kernel limits how many a process can hold.

Inspecting Open Files

```
# Exact number of open descriptors for a process
sudo ls /proc/1234/fd | wc -l

# What those descriptors are
sudo lsof -p 1234

# Which process has a file open
sudo lsof /var/log/application.log

# Which process is using a port
sudo lsof -i :8080
```

INTERVIEW TRAP

`lsof -p PID | wc -l` overcounts, because lsof also lists memory-mapped libraries, the working directory and a header line. Use `ls /proc/PID/fd | wc -l` for the real descriptor count.

Per-Process Limits

```
# Current shell's open-file limit, and all limits
ulimit -n
ulimit -a

# The limits actually applied to a running process
cat /proc/1234/limits
```

Soft limit	Hard limit
The limit currently enforced	The ceiling the soft limit can be raised to
A user can raise it up to the hard limit	Only root can raise it

`ulimit -n 65535` affects only the current shell and processes started from it, and only if the hard limit allows it.

INTERVIEW TRAP

Your shell's `ulimit -n` does not show a service's limit. A systemd service gets its limit from `LimitNOFILE=` in its unit file (see Chapter 5). Always check `/proc/PID/limits` for the process that is failing.

System-Wide Limits

```
# Maximum file handles for the whole system
cat /proc/sys/fs/file-max

# Allocated, unused, maximum
cat /proc/sys/fs/file-nr

# Number of processes owned by a user
ps -u appuser --no-headers | wc -l
```

Error	Meaning
`EMFILE` ("Too many open files")	This **process** hit its own open-file limit
`ENFILE` ("Too many open files in system")	The **whole system** hit `fs.file-max`

10 Kernel Parameters with sysctl

`sysctl` reads and changes runtime kernel parameters stored under `/proc/sys`. Chapter 8 covered the basics and common DevOps parameters.

```
# Read one parameter
sysctl net.ipv4.ip_forward

# Change it until the next reboot
sudo sysctl -w net.ipv4.ip_forward=1

# Make it permanent and apply
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-custom.conf
sudo sysctl --system
```

Parameter	Purpose
`vm.swappiness`	Tendency to use swap
`net.ipv4.ip_forward`	IPv4 packet forwarding (needed by Docker and Kubernetes)
`fs.file-max`	System-wide maximum file handles
`kernel.pid_max`	Maximum PID value, which also limits the total number of processes and threads

IMPORTANT

Kernel parameters affect the entire server. Understand the workload and test changes before applying them in production.

11 Tracing System Calls with strace

`strace` shows every system call a process makes and what the kernel returned. It is extremely useful when an application fails but its logs do not explain why.

```
# Trace a command
strace ls

# Attach to a running process
sudo strace -p 1234

# Also follow child processes and threads (important for services)
sudo strace -f -p 1234

# Only file-related or network-related calls
strace -e trace=file ls
strace -e trace=network curl -s http://localhost

# Save output to a file
strace -o trace.log ls

# Summary of which calls were made and how long they took
strace -c ls
```

Investigating a Permission Problem

```
sudo strace -f -e trace=openat -p 1234
```

```
openat(AT_FDCWD, "/etc/app/config.conf", O_RDONLY) = -1 EACCES (Permission denied)
```

The process was denied access to that exact file. Possible causes: wrong permissions or ownership, missing execute permission on a parent directory, or SELinux/AppArmor.

Common Error Codes

Error	Meaning
`EACCES`	Permission denied
`ENOENT`	File or directory does not exist
`EMFILE`	Process reached its open-file limit
`ENFILE`	System-wide file table limit reached
`ENOMEM`	Not enough memory for the operation
`ECONNREFUSED`	Nothing is listening on the target port

WARNING

`strace` slows down the traced process significantly and can produce huge output. On production, trace only specific calls (`-e trace=`), keep it short, and never leave it attached.

12 PID 1 and the Process Tree

After booting, the kernel starts the first user-space process, PID 1. On most modern distributions it is systemd (see Chapter 8).

```
ps -p 1 -o pid,comm,args
pstree -p | head
```

PID 1 starts and supervises services, handles startup targets and shutdown, and adopts and reaps orphaned processes.

INTERVIEW NOTE

Inside a container, PID 1 is usually the application itself, not systemd. This changes how signals and zombie processes are handled, which is covered in Chapter 15.

13 Practical Troubleshooting Scenarios

Scenario 1 Application reports "Too many open files"

```
# The service's real limit
cat /proc/1234/limits | grep "open files"

# How many it has open right now
sudo ls /proc/1234/fd | wc -l

# What they are
sudo lsof -p 1234 | awk '{print $5}' | sort | uniq -c | sort -nr
```

If the count keeps growing, the application is leaking descriptors (often unclosed sockets). If the workload genuinely needs more, raise `LimitNOFILE=` in the unit file (see Chapter 5).

Scenario 2 Permission denied, but permissions look correct

```
# Which exact file is being denied?
sudo strace -f -e trace=openat -p 1234

# Check permissions of every directory in the path
namei -l /etc/app/config.conf

# Check the user the process runs as
ps -o user,pid,cmd -p 1234
```

`namei -l` often reveals a parent directory without execute permission. If Unix permissions are correct, check SELinux or AppArmor (see Chapter 9).

Scenario 3 Cannot create files despite free disk space

```
df -h
df -i
```

If inode usage is 100%, the filesystem has run out of inodes because of many small files. Disk space and inodes are separate resources (see Chapters 2 and 6).

Scenario 4 A required kernel module is missing

```
# Is it loaded?
lsmod | grep overlay

# Is it available for this kernel?
modinfo overlay

# Load it, then check kernel logs for driver errors
sudo modprobe overlay
journalctl -k -n 50
```

If `modinfo` cannot find it, the module may not be installed for the running kernel, for example after a kernel upgrade without a reboot or missing extra-modules packages.

Scenario 5 Application fails with an unclear error

```
sudo strace -f -o /tmp/trace.log -p 1234
grep -E "= -1 E" /tmp/trace.log | tail -20
```

The failing system calls and their error codes show whether the problem is file access, network connections, resource limits or memory.

14 Linux Internals Cheat Sheet

Requirement	Command
Kernel version	`uname -r`
Process tree with PIDs	`pstree -p`
PID, PPID, PGID, SID	`ps -o pid,ppid,pgid,sid,cmd -p PID`
Threads of a process	`ps -T -p PID`
Context switches	`vmstat 1`, `grep ctxt /proc/PID/status`
CPU affinity	`taskset -cp PID`
Loaded modules / details	`lsmod`, `modinfo NAME`
Load a module	`sudo modprobe NAME`
Process limits	`cat /proc/PID/limits`
Open descriptor count	`ls /proc/PID/fd | wc -l`
Open files of a process	`lsof -p PID`
System-wide file handles	`cat /proc/sys/fs/file-nr`
Kernel parameter	`sysctl NAME`
Permissions along a path	`namei -l /path/to/file`
Trace a process and its children	`strace -f -p PID`
Kernel logs	`journalctl -k`

15 Best Practices

1. **Know which layer fails:** a permission error can come from file permissions, a parent directory, SELinux/AppArmor or a resource limit.
2. **Check the real process limits** in `/proc/PID/limits`, not your shell's `ulimit`.
3. **Use strace with filters** and for a short time only.
4. **Avoid unnecessary kernel changes:** test module and sysctl changes before production.
5. **Use process relationships** (`pstree`, PPID, PGID) to find stray background processes.
6. **Use /proc and /sys** for live diagnostic information.

7. **Fix descriptor leaks** instead of only raising limits.
8. **Collect evidence** before restarting or changing anything.

16 Common Interview Questions

Q1. What is a system call?

The interface a user-space program uses to request services from the kernel, such as opening files, creating processes or using the network.

Q2. What is the difference between user space and kernel space?

User space runs applications with restricted privileges. Kernel space runs the kernel with full hardware access and performs privileged operations on behalf of applications.

Q3. What is the difference between fork() and exec()?

`fork()` creates a child process as a copy of the parent. `exec()` replaces the current program with a new one while keeping the same PID.

Q4. What is copy-on-write?

After `fork()`, parent and child share memory pages until one of them writes. Only then is the page copied. This makes process creation fast.

Q5. Why does a parent need to call wait()?

To collect the child's exit status. Otherwise the finished child stays in the process table as a zombie.

Q6. What is a file descriptor?

A small integer a process uses to refer to an open file, socket, pipe or device. 0, 1 and 2 are stdin, stdout and stderr.

Q7. What is context switching, and what do voluntary vs involuntary switches mean?

Switching the CPU from one task to another by saving and restoring state. Voluntary switches happen when a task waits; involuntary switches happen when the scheduler preempts it, and high counts suggest CPU contention.

Q8. What is a kernel module?

Code that can be loaded into or removed from the running kernel, such as a hardware driver or filesystem support, without rebuilding the kernel.

Q9. What is the difference between modprobe and insmod?

`modprobe` loads a module by name and resolves its dependencies. `insmod` inserts a specific module file and does not handle dependencies.

Q10. What is the difference between /proc and /sys?

`/proc` exposes process information and kernel state. `/sys` exposes devices, drivers and kernel objects. Both are virtual and generated by the kernel.

Q11. What is the difference between EMFILE and ENFILE?

`EMFILE` means one process hit its own open-file limit. `ENFILE` means the whole system hit its file-handle limit.

Q12. What is the difference between soft and hard limits?

The soft limit is currently enforced. The hard limit is the ceiling to which the soft limit can be raised; only root can raise the hard limit.

Q13. Why does `ulimit -n` in your shell not match a service's limit?

`ulimit` shows the current shell's limit. A systemd service gets its limit from `LimitNOFILE=` in its unit file. Check `/proc/PID/limits` for the real value.

Q14. What is strace used for?

It traces the system calls and signals of a process. It is useful for finding exactly which file, connection or resource causes a failure that logs do not explain.

Q15. What is CPU affinity?

A setting that restricts which CPU cores a process may run on, managed with `taskset`.

Q16. What is PID 1?

The first user-space process in a PID namespace. On most systems it is systemd; it starts services and adopts orphaned processes. In containers, it is usually the application.

17 Chapter Takeaways

After completing this chapter, you should understand:

- ✓ Kernel architecture and the system call interface.
- ✓ How applications use system calls and file descriptors.
- ✓ How `fork()`, `exec()` and `wait()` create and clean up processes, including copy-on-write.
- ✓ Process groups, sessions and parent-child relationships.
- ✓ Threads, voluntary and involuntary context switches.
- ✓ Scheduling, nice values and CPU affinity.
- ✓ Kernel modules and `modprobe` vs `insmod`.
- ✓ The `/proc` and `/sys` virtual filesystems.
- ✓ File descriptor limits, EMFILE vs ENFILE, and soft vs hard limits.
- ✓ Kernel parameters with `sysctl`.
- ✓ System-call tracing with `strace`.
- ✓ The role of PID 1 on hosts and in containers.

PRACTICAL EXERCISES

Perform these on a Linux test machine.

1. Display the kernel version using `uname -r`.
2. Identify PID 1 and view the process tree with `pstree -p`.
3. Find the PPID, PGID and SID of your current shell.
4. List all threads of a running process.
5. Compare voluntary and involuntary context switches for a process.
6. Check the CPU affinity of a process using `taskset -cp`.
7. List loaded kernel modules and inspect one with `modinfo`.
8. Read CPU and memory information from `/proc`.
9. List network interfaces from `/sys/class/net/`.
10. Compare `ulimit -n` with `/proc/PID/limits` for a running service.
11. Count a process's open descriptors using `/proc/PID/fd`.
12. Check system-wide file handle usage with `/proc/sys/fs/file-nr`.
13. Trace a simple command with `strace -e trace=file`.
14. Create a permission-denied error and investigate it with `strace` and `namei -l`.

INTERVIEW TARGET

Be able to explain how applications use system calls, how processes are created, how file descriptors and limits work, and how to use /proc, sysctl, lsof and strace to investigate low-level Linux problems.

NEXT CHAPTER

Chapter 15: Linux for Docker and Kubernetes

You will learn how Linux namespaces, cgroups, OverlayFS and capabilities power containers, and how to troubleshoot container issues from the underlying Linux host.

END OF CHAPTER 14